9. Greedy Algorithms & Heaps – Comprehensive Material

Module 1: Greedy Algorithms – Core Concepts

1.1 What is a Greedy Algorithm?

A greedy algorithm builds a solution step by step, always picking the **locally optimal** choice at each step, hoping that these local optima lead to a **global optimum**.

Characteristics:

- **Greedy choice property:** A global optimum can be reached by making a locally optimal choice.
- **Optimal substructure:** An optimal solution to the problem contains optimal solutions to subproblems.

When to use: Problems where choosing the “best” option now never prevents a better solution later (e.g., fractional knapsack, activity selection).

Common greedy problems:

- Activity Selection
- Fractional Knapsack
- Job Sequencing with Deadlines
- Huffman Coding
- Minimum Spanning Tree (Prim’s, Kruskal’s)

- Dijkstra's Shortest Path

Module 2: Activity Selection – Greedy vs DP

2.1 Problem Statement

Given `n` activities with start and end times, select the **maximum number** of non-overlapping activities. Each activity can be represented as `(start, end)`.

2.2 Greedy Solution (Optimal)

Strategy: Sort activities by **end time**. Always pick the activity that ends earliest, then skip overlapping ones.

Code:

```
import java.util.*;

class Activity {
    int start, end;
    Activity(int s, int e) { start = s; end = e; }
}

public class ActivitySelection {
    public static List<Activity> selectActivities(List<Ac
        // Sort by end time
        Collections.sort(activities, Comparator.comparing
        List<Activity> result = new ArrayList<>();
        if (activities.isEmpty()) return result;

        int lastEnd = activities.get(0).end;
        result.add(activities.get(0));
```

```
        for (int i = 1; i < activities.size(); i++) {
            Activity a = activities.get(i);
            if (a.start >= lastEnd) {
                result.add(a);
                lastEnd = a.end;
            }
        }
        return result;
    }
}
```

Complexity: O(n log n) for sorting, O(n) for selection.

2.3 Comparison: Greedy vs Brute-Force (Non-Greedy)

A brute-force solution would consider all subsets of activities and check which subset is non-overlapping and has maximum size – $O(2^n)$ time. A DP solution (like weighted interval scheduling) would also be O(n log n) but with more overhead.

Why Greedy works: By picking the activity that ends earliest, we leave the maximum possible room for remaining activities. This is the **greedy choice property** – any optimal solution can be transformed to include this earliest-finishing activity without reducing the count.

Correctness argument (for hands-on): We can prove by induction or exchange argument. For any optimal solution, if it doesn't include the earliest-ending activity, we can replace the first activity with it without affecting feasibility and without decreasing count.

2.4 Programmatic Comparison (Simulation)

We can write a small test that runs both greedy and exhaustive search for small `n` to verify greedy gives the same result. Below is a pseudo-code for exhaustive search (not shown in full) – but the point is that greedy is **much faster**.

Module 3: Fractional Knapsack – Greedy vs 0/1 Knapsack

3.1 Problem Statement

Given items with weight and value, and a knapsack capacity, maximize total value. In **fractional knapsack**, items can be broken into fractions. In **0/1 knapsack**, items must be taken whole.

3.2 Fractional Knapsack – Greedy

Strategy: Sort items by **value/weight ratio** descending. Take as much as possible of the highest-ratio item, then the next, etc.

Code:

```
class Item { int weight, value; }

public static double fractionalKnapsack(int capacity, Lis
    Collections.sort(items, (a,b) -> Double.compare(
        (double)b.value / b.weight, (double)a.value / a.w
    ));
    double totalValue = 0;
    int remaining = capacity;
    for (Item item : items) {
        if (remaining <= 0) break;
        if (item.weight <= remaining) {
            totalValue += item.value;
            remaining -= item.weight;
        } else {
            totalValue += (double)item.value * remaining
            remaining = 0;
        }
    }
    return totalValue;
}
```

Complexity: O(n log n) for sorting.

3.3 Why Greedy Works for Fractional

Since we can take fractions, we always take the item with the highest value density – this is optimal. We can also compare with **0/1 Knapsack** (DP) which is O(n×capacity) and does not accept greedy as optimal.

Programmatic comparison: On the same input, fractional greedy gives optimal (and usually higher or equal value than any feasible 0/1 solution). We can write a driver that shows both outputs and explains that fractional knapsack cannot be solved by greedy for 0/1.

Module 4: Job Sequencing with Deadlines

4.1 Problem Statement

Given `n` jobs, each with a deadline `d` and profit `p`. Each job takes 1 unit of time. Schedule jobs to maximize total profit while meeting deadlines.

4.2 Greedy Solution

Strategy: Sort jobs by **profit descending**. For each job, assign it to the latest available slot before its deadline (using a disjoint set or simple array). This greedy works because we prioritize high-profit jobs.

Code (using array for time slots, O(n²) naive):

```
class Job { int id, deadline, profit; }

public static int jobScheduling(List<Job> jobs) {
    Collections.sort(jobs, (a,b) -> b.profit - a.profit);
    int maxDeadline = 0;
```

```
    for (Job j : jobs) maxDeadline = Math.max(maxDeadline
    int[] slot = new int[maxDeadline + 1]; // 0 = free
    int totalProfit = 0;
    for (Job j : jobs) {
        // find a free slot from deadline down to 1
        for (int t = j.deadline; t >= 1; t--) {
            if (slot[t] == 0) {
                slot[t] = j.id;
                totalProfit += j.profit;
                break;
            }
        }
    }
    return totalProfit;
}
```

Optimized using DSU (Union-Find) can achieve O(n log n + n α(n)).

Correctness: Greedy works because if a job with high profit cannot be scheduled in its slot, no other job with lower profit should take that slot. Exchange argument: any optimal schedule can be adjusted to include the highest-profit job without reducing total profit.

Module 5: Interval Merging (Greedy Sorting)

5.1 Problem Statement

Given a collection of intervals, merge all overlapping intervals.

5.2 Greedy Approach

Strategy: Sort intervals by start time. Iterate and merge if the next interval overlaps with the current merged one.

Code:

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a,b) -> a[0] - b[0]);
    List<int[]> merged = new ArrayList<>();
    int[] current = intervals[0];
    merged.add(current);
    for (int[] interval : intervals) {
        if (interval[0] <= current[1]) {
            current[1] = Math.max(current[1], interval[1]
        } else {
            current = interval;
            merged.add(current);
        }
    }
    return merged.toArray(new int[merged.size()][]);
}
```

Why Greedy: Sorting by start time ensures that we process intervals in order; merging overlapping ones greedily gives the correct set of merged intervals.

Module 6: Huffman Coding – Concept

Problem: Given characters with frequencies, assign variable-length binary codes to minimize total bits. Greedy approach: repeatedly merge two least frequent nodes.

Algorithm:

1. Create a min-heap of nodes (each char with its frequency).
2. While more than one node: extract two with smallest frequencies, create a new internal node with sum frequency, push back.
3. The resulting tree gives Huffman codes.

Why Greedy: By always merging the two smallest frequencies, we

ensure the least frequent characters get the longest codes (or shortest, depending on tree structure), which is optimal for prefix-free codes.

(Code not shown, but concept is well-known.)

Module 7: Heaps – Min, Max, Top-K, Running Median

7.1 Min-Heap and Max-Heap in Java

Java's `PriorityQueue` is a min-heap by default. For max-heap, use `Collections.reverseOrder()`.

```
// Min-heap
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
// Max-heap
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Coll
```

7.2 Top-K Problems

Kth Largest Element: Use a min-heap of size `k`. Iterate over elements; if heap size < k, add; else if current > heap.peek(), replace.

Code:

```
public int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>();
    for (int num : nums) {
        heap.offer(num);
        if (heap.size() > k) heap.poll();
    }
    return heap.peek();
}
```

7.3 Running Median using Two Heaps

Maintain a **max-heap** for the lower half and a **min-heap** for the upper half. Rebalance to keep sizes equal or max-heap has one extra.

Code:

```
class MedianFinder {
    PriorityQueue<Integer> maxHeap; // lower half (smalle
    PriorityQueue<Integer> minHeap; // upper half (larger
    public MedianFinder() {
        maxHeap = new PriorityQueue<>(Collections.reverse
        minHeap = new PriorityQueue<>();
    }
    public void addNum(int num) {
        if (maxHeap.isEmpty() || num <= maxHeap.peek()) {
            maxHeap.offer(num);
        } else {
            minHeap.offer(num);
        }
        // rebalance
        if (maxHeap.size() > minHeap.size() + 1) {
            minHeap.offer(maxHeap.poll());
        } else if (minHeap.size() > maxHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
    }
    public double findMedian() {
        if (maxHeap.size() == minHeap.size()) {
            return (maxHeap.peek() + minHeap.peek()) / 2.
        } else {
            return maxHeap.peek();
        }
    }
}
```

Complexity: O(log n) per add, O(1) for median.

Module 8: Case Study – Conference Room Booking (Interval Scheduling)

Scenario: A company has multiple meeting rooms. Given a list of meeting intervals (start, end), determine the **minimum number of rooms** needed to host all meetings.

Greedy solution: Sort meetings by start time. Use a min-heap (priority queue) to store end times of rooms currently in use. For each meeting, if the earliest available room (smallest end) is free (end ≤ current start), reuse it (poll and then offer new end). Otherwise, allocate a new room.

Code:

```
public int minMeetingRooms(int[][] intervals) {
    if (intervals.length == 0) return 0;
    Arrays.sort(intervals, (a,b) -> a[0] - b[0]);
    PriorityQueue<Integer> rooms = new PriorityQueue<>();
    rooms.offer(intervals[0][1]);
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] >= rooms.peek()) {
            rooms.poll(); // reuse room
        }
        rooms.offer(intervals[i][1]);
    }
    return rooms.size();
}
```

Greedy correctness: Sorting by start and always reusing the room that becomes free earliest is optimal (exchange argument).

Module 9: Hands-on Problems – Solutions

9.1 Minimum Platforms (GeeksforGeeks)

Problem: Given arrival and departure times of trains, find the minimum number of platforms needed.

Greedy: Sort arrivals and departures separately. Use two pointers; when a train arrives before the earliest departure, increase platform count; else decrease.

Code:

```
public static int minPlatforms(int[] arr, int[] dep) {
    Arrays.sort(arr);
    Arrays.sort(dep);
    int platforms = 0, maxPlatforms = 0;
    int i = 0, j = 0;
    while (i < arr.length && j < dep.length) {
        if (arr[i] <= dep[j]) {
            platforms++;
            i++;
            maxPlatforms = Math.max(maxPlatforms, platfor
        } else {
            platforms--;
            j++;
        }
    }
    return maxPlatforms;
}
```

9.2 Jump Game (LeetCode 55)

Problem: Given an array, each element is the maximum jump length from that position. Determine if you can reach the last index.

Greedy: Keep track of the farthest reachable index. Iterate, if current index > farthest, return false; else update farthest = max(farthest, i + nums[i]).

Code:

```
public boolean canJump(int[] nums) {
    int farthest = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > farthest) return false;
        farthest = Math.max(farthest, i + nums[i]);
        if (farthest >= nums.length - 1) return true;
    }
    return true;
}
```

9.3 Job Sequencing (see Module 4)

9.4 Activity Selection (see Module 2)

9.5 Greedy Correctness Argument (for Activity Selection)

We can argue by induction: For the earliest-finishing activity `a1`, there exists an optimal solution that includes `a1`. Take any optimal solution; if it contains `a1`, done. If not, let `a_k` be the first activity in that optimal solution. Since `a1` ends no later than `a_k`, we can replace `a_k` with `a1` without causing overlap (because `a1` ends earlier, it frees up even more time). Thus, an optimal solution exists with `a1`. Then recursively solve the subproblem on activities that start after `a1` ends. This proves greedy optimal.

Module 10: Programmatic Comparison – Greedy vs Non-Greedy (with Examples)

10.1 Activity Selection – Greedy vs Exhaustive Search

We can simulate the exhaustive search for small `n` to confirm greedy gives the same max count:

```
// Exhaustive (recursive) for maximum non-overlapping act
public static int maxActivities(List<Activity> activities
    // pseudo-code
}
```

But practically, we show that greedy runs in O(n log n) whereas exhaustive is exponential.

10.2 Fractional Knapsack – Greedy vs 0/1 DP

Write both and show outputs on a sample input, highlighting that greedy fails for 0/1 but works for fractional.

Sample input: Capacity = 10 Items: (weight=6, value=30), (weight=5, value=20), (weight=5, value=20) Greedy fractional: take full of first (ratio 5), then 4/5 of second → total value = 30 + 16 = 46 (or actually take 30 + 20*4/5? Let's compute properly: ratios: item1=5, item2=4, item3=4 → take item1 (6kg, 30) then remaining 4kg of item2 -> value 16 → total 46. Optimal fractional = 46. 0/1 knapsack DP: choose item2+item3 (10kg, 40) or item1+? can't fit second (5+6>10) so max 40. Greedy would also pick item1 first then cannot pick any other because weight left 4, so only 30. So greedy fails for 0/1. This demonstrates the difference.

Practice Exercises (10 problems)

1. **Activity Selection** – Implement and test with random intervals.
2. **Fractional Knapsack** – Write greedy solution.
3. **Job Sequencing with Deadlines** – Use DSU for optimization.
4. **Minimum Platforms** – Solve with sorting.
5. **Jump Game II** (LeetCode 45) – Greedy with min jumps.
6. **Merging Intervals** – Implement and test.
7. **Huffman Coding** – Build the tree and generate codes.

8. **Kth Largest Element in a Stream** (LeetCode 703) – Use min-heap.
9. **Find Median from Data Stream** – Two heaps.
10. **Conference Room Booking** (Min Rooms) – Heap approach.

Summary Table

Problem	Greedy Strategy	Data Structure	Complexity
Activity Selection	Sort by end time	Array/list	O(n log n)
Fractional Knapsack	Sort by value/ weight	Array	O(n log n)
Job Sequencing	Sort by profit, assign to latest slot	Array / DSU	O(n log n)
Interval Merging	Sort by start	Array	O(n log n)
Min Platforms	Sort arrivals/ departures	Two pointers	O(n log n)
Jump Game	Keep farthest reachable	–	O(n)
Running Median	Two heaps	max-heap & min-heap	O(log n) per add
Top-K	Min-heap of size K	PriorityQueue	O(n log K)